Telegram Mini Apps

Fonte: https://core.telegram.org/bots/webapps · Extraído em 12 de outubro de 2026

With **Mini Apps** developers can use *JavaScript* to create **infinitely flexible interfaces** that can be launched right inside Telegram — and can completely replace **any website**.

Like bots, **Mini Apps** support seamless authorization, payments via third-party payment providers (with *Google Pay* and *Apple Pay* out of the box), delivering tailored push notifications to users, and much more.

> To see a **Mini App** in action, try our sample @DurgerKingBot.

Recent changes

June 11, 2026

Bot API 10.1

- Added the field *chat_join_request_query_id* to the class WebAppInitData.

April 3, 2026

Bot API 9.6

- Added the method *requestChat* to the class WebApp.

March 1, 2026

Bot API 9.5

- Added the field *iconCustomEmojiId* to the class BottomButton.

July 3, 2025

Bot API 9.1

- Added the method *hideKeyboard* to the class WebApp.

April 11, 2025

Bot API 9.0

- Added the field DeviceStorage, allowing Mini Apps to use persistent local storage on the user's device.
- Added the field SecureStorage, allowing Mini Apps to use a secure local storage on the user's device for sensitive data.

November 17, 2024

Bot API 8.0

> This is the **largest update** in the history of Telegram mini apps – adding more than **10 new features** and monetization options for developers. To read more about all these changes, check out this dedicated blog post.

Full-screen Mode

- Mini Apps are now able to become full-screen in both portrait and **landscape mode** – allowing them to host **more games**, play **widescreen media** and support **immersive** user experiences.
- Added the methods *requestFullscreen* and *exitFullscreen* to toggle full-screen mode.
- Added the fields *safeAreaInset* and *contentSafeAreaInset*, allowing Mini Apps to ensure that their content properly respects the device's safe area margins.
- Further added the fields *isActive* and *isFullscreen* to the class WebApp.
- Added the events *activated*, *deactivated*, *safeAreaChanged*, *contentSafeAreaChanged*, *fullscreenChanged* and *fullscreenFailed*.

Homescreen Shortcuts

- Mini Apps can now be accessed via direct shortcuts added to the **home screen** of mobile devices.
- Added the method *addToHomeScreen* to create a shortcut for users to add to their home screens.
- Added the method *checkHomeScreenStatus* to determine the status and support of the home screen shortcut for the Mini App on the current device.
- Added the events *homeScreenAdded* and *homeScreenChecked*.

Emoji Status

- Mini Apps can now prompt users to set their emoji status – or request access to later sync it automatically with in-game badges, third-party APIs and more.
- Added the method *setEmojiStatus* to let users manually confirm a custom emoji as their new status via a native dialog.
- Added the method *requestEmojiStatusAccess* for obtaining permission to later update a user's emoji status via the Bot API method setUserEmojiStatus.
- Added the events *emojiStatusSet*, *emojiStatusFailed* and *emojiStatusAccessRequested*.

Media Sharing and File Downloads

- Users can now share media directly from Mini Apps – sending **referral codes**, custom memes, artwork and more to **any chat** or posting them as a story.
- Added the method *shareMessage* to share media from Mini Apps to Telegram chats. Also see PreparedInlineMessage.
- Added the method *downloadFile*, introducing support for a **native popup** that prompts users to download files from the Mini App.
- Added the events *shareMessageSent*, *shareMessageFailed* and *fileDownloadRequested*.

Geolocation Access

- Mini Apps can now request geolocation access to users, allowing them to build virtually any location-based service, from **games** with dynamic points of interest to **interactive maps** for events.

- Added the field *LocationManager* to the class WebApp.
- Added the events *locationManagerUpdated* and *locationRequested*.

Device Motion Tracking

- Mini Apps can now track detailed device motion data, allowing them to implement better productivity tools, immersive **VR experiences** and more.
- Added the fields *isOrientationLocked*, *Accelerometer*, *DeviceOrientation* and *Gyroscope* to the class WebApp.
- Added the methods *lockOrientation* and *unlockOrientation* to control the screen orientation.
- Added the events *accelerometerStarted*, *accelerometerStopped*, *accelerometerChanged*, *accelerometerFailed*, *deviceOrientationStarted*, *deviceOrientationStopped*, *deviceOrientationChanged*, *deviceOrientationFailed*, *gyroscopeStarted*, *gyroscopeStopped*, *gyroscopeChanged*, *gyroscopeFailed*.

Subscription Plans and Gifts for Telegram Stars

- Mini Apps now support **paid subscriptions** powered by Telegram Stars – **monetizing their efforts** with multiple tiers of content and features.
- Mini Apps can use their balance of Telegram Stars to **send gifts** to their users.
- You can read more on implementing Paid Subscriptions and Gifts in our Bot API documentation.

Loading Screen Customization

- Mini Apps can customize their loading screen, adding **their own icon** and **specific colors** for light and dark themes.
- You can access these customization settings in @BotFather via */mybots* > *Select Bot* > *Bot Settings* > *Configure Mini App* > *Enable Mini App*

Hardware-specific Optimizations

- Mini Apps running on Android can now receive basic information about a device's processing hardware, allowing them to **optimize user experience** based on the device's capabilities.
- This information includes the OS, App and SDK's respective versions as well as the device's model and performance class.

General

- The field *photo_url* in the class WebAppUser is now available to all Mini Apps, allowing them to access a user's profile photo if their privacy settings allow for it.
- Third parties (e.g., Mini App builders, external SDKs etc.) that receive or process data on behalf of Mini Apps are now able to validate it without knowing the App's bot token.
- Debugging options have been expanded to include full support for **iOS devices**. You can use these tools to find app-specific issues in your Mini App.

September 6, 2024

Bot API 7.10

- Added the field *SecondaryButton* to the class WebApp.
- Added the event *secondaryButtonClicked*.
- Renamed the class *MainButton* to the class BottomButton.
- Added the field *bottomBarColor* and the method *setBottomBarColor* to the class WebApp.

- Added the field *bottom_bar_bg_color* to the class ThemeParams.

July 31, 2024

Bot API 7.8

- Added the option for bots to set a Main Mini App, which can be previewed and launched directly from a button in the bot's profile or a link.
- Added the method *shareToStory* to the class WebApp.

July 7, 2024

Bot API 7.7

- Added the field *isVerticalSwipesEnabled* and the methods *enableVerticalSwipes*, *disableVerticalSwipes* to the class WebApp.
- Added the event *scanQrPopupClosed*.

July 1, 2024

Bot API 7.6

- Added the field *section_separator_color* to the class ThemeParams.
- Changed the default opening mode for Direct Link Mini Apps.

March 31, 2024

Bot API 7.2

- Added the field *BiometricManager* to the class WebApp.

December 29, 2023

Bot API 7.0

- Added the field *SettingsButton* to the class WebApp.
- Added the fields *header_bg_color*, *accent_text_color*, *section_bg_color*, *section_header_text_color*, *subtitle_text_color*, *destructive_text_color* to the class ThemeParams.
- Mini Apps no longer close when the method *WebApp.openTelegramLink* is called.

September 22, 2023

Bot API 6.9

- Added the field *CloudStorage* to the class WebApp.
- Added the methods *requestWriteAccess* and *requestContact* to the class WebApp.
- Added the fields *added_to_attachment_menu* and *allows_write_to_pm* to the class WebAppUser.
- Added the events *writeAccessRequested* and *contactRequested*.
- Added the ability to set any header color using the *setHeaderColor* method.

April 21, 2023

Bot API 6.7

- Added support for launching Mini Apps from inline query results and from a direct link.
- Added the method *switchInlineQuery* to the class WebApp.

December 30, 2022

Bot API 6.4

- Added the field *platform*, the optional parameter *options* to the method *openLink* and the methods *showScanQrPopup*, *closeScanQrPopup*, *readTextFromClipboard* to the class WebApp.
- Added the events *qrTextReceived*, *clipboardTextReceived*.

August 12, 2022

Bot API 6.2

- Added the field *isClosingConfirmationEnabled* and the methods *enableClosingConfirmation*, *disableClosingConfirmation*, *showPopup*, *showAlert*, *showConfirm* to the class WebApp.
- Added the field *is_premium* to the class WebAppUser.
- Added the event *popupClosed*.

June 20, 2022

Bot API 6.1

- Added the ability to use bots added to the attachment menu in group, supergroup and channel chats.
- Added support for t.me links that can be used to select the chat in which the attachment menu with the bot will be opened.
- Added the fields *version*, *headerColor*, *backgroundColor*, *BackButton*, *HapticFeedback* and the methods *isVersionAtLeast*, *setHeaderColor*, *setBackgroundColor*, *openLink*, *openTelegramLink*, *openInvoice* to the class WebApp.
- Added the field *secondary_bg_color* to the class ThemeParams.
- Added the method *offClick* to the class MainButton.
- Added the fields *chat*, *can_send_after* to the class WebAppInitData.
- Added the events *backButtonClicked*, *settingsButtonClicked*, *invoiceClosed*.

Designing Mini Apps

Color Schemes

Mini Apps always receive data about the user's current **color theme** in real time, so you can adjust the appearance of your interfaces to match it. For example, when users switch between **Day and Night** modes or use various custom themes.

> Jump to technical information

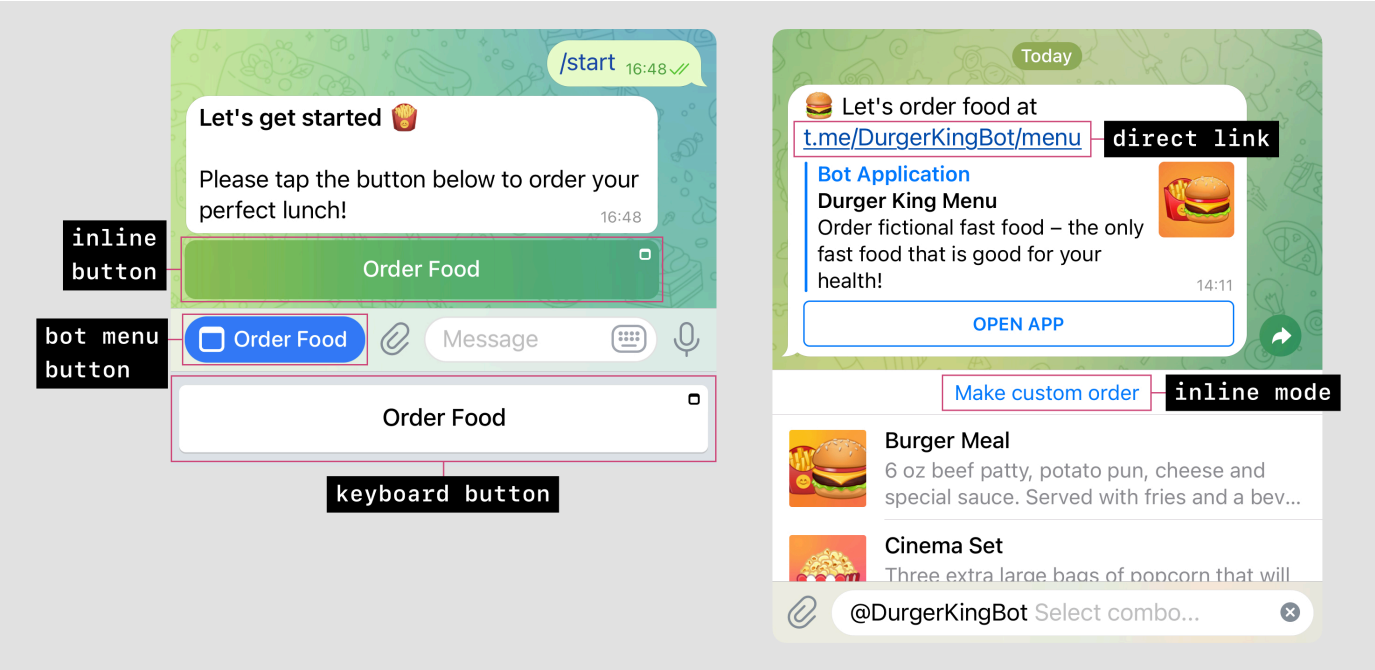
Design Guidelines

Telegram apps are known for being snappy, smooth and following a consistent cross-platform design. Your Mini App should ideally reflect these principles.

- All elements should be responsive and designed with a mobile-first approach.
- Interactive elements should mimic the style, behavior, and intent of UI components that already exist.
- All included animations should be smooth, ideally 60fps.
- All inputs and images should contain labels for accessibility purposes.
- The app should deliver a seamless experience by monitoring the dynamic theme-based colors provided by the API and using them accordingly.
- Ensure that the app's interface respects the safe area and content safe area to avoid overlapping with control elements, especially when using fullscreen mode.
- For Android devices, consider the additional information in the User-Agent (see User-Agent details) and adjust for the device's performance class, minimizing animations and visual effects on low-performance devices to ensure smooth performance.

Implementing Mini Apps

Telegram currently supports seven different ways of launching Mini Apps: the main Mini App from a profile button, from a keyboard button, from an inline button, from the bot menu button, via inline mode, from a direct link – and even from the attachment menu.



Keyboard Button Mini Apps

> **TL;DR:** Mini Apps launched from a **web_app** type keyboard button can send data back to the bot in a *service message* using Telegram.WebApp.sendData. This makes it possible for the bot to produce a response without communicating with any external servers.

Users can interact with bots using custom keyboards, buttons under bot messages, as well as by sending freeform **text messages** or any of the **attachment types** supported by Telegram: photos and videos, files, locations, contacts and polls. For even more flexibility, bots can utilize the full power of **HTML5** to create user-friendly input interfaces.

You can send a **web_app** type KeyboardButton that opens a Mini App from the specified URL.

To transmit data from the user back to the bot, the Mini App can call the Telegram.WebApp.sendData method. Data will be transmitted to the bot as a String in a service message. The bot can continue communicating with the user after receiving it.

Good for:

- **Custom data input interfaces** (a personalized calendar for selecting dates; selecting data from a list with advanced search options; a randomizer that lets the user “spin a wheel” and chooses one of the available options, etc.)
- **Reusable components** that do not depend on a particular bot.

Inline Button Mini Apps

> **TL;DR:** For more interactive Mini Apps like @DurgerKingBot, use a **web_app** type Inline KeyboardButton, which gets basic user information and can be used to send a message on behalf of the user to the chat with the bot.

If receiving text data alone is insufficient or you need a more advanced and personalized interface, you can open a Mini App using a **web_app** type Inline KeyboardButton.

From the button, a Mini App will open with the URL specified in the button. In addition to the user's theme settings, it will receive basic user information (`ID` , `name` , `username` , `language_code`) and a unique identifier for the session, **query_id**, which allows messages on behalf of the user to be sent back to the bot.

The bot can call the Bot API method answerWebAppQuery to send an inline message from the user back to the bot and close the Mini App. After receiving the message, the bot can continue communicating with the user.

Good for:

- Fully-fledged web services and integrations of any kind.
- The use cases are effectively **unlimited**.

Launching Mini Apps from the Menu Button

> **TL;DR:** Mini Apps can be launched from a customized menu button. This simply offers a quicker way to access the app and is otherwise **identical** to launching a mini app from an inline button.

By default, chats with bots always show a convenient **menu button** that provides quick access to all listed commands. With Bot API 6.0, this button can be used to **launch a Mini App** instead.

To configure the menu button, you must specify the text it should show and the Mini App URL. There are two ways to set these parameters:

- To customize the button for **all users**, use @BotFather (the `/setmenubutton` command or *Bot Settings > Menu Button*).
- To customize the button for both **all users** and **specific users**, use the setChatMenuButton method in the Bot API. For example, change the button text according to the user's language, or show links to different Mini Apps based on a user's settings in your bot.

Apart from this, Mini Apps opened via the menu button work in the exact same way as when using inline buttons.

> @DurgerKingBot allows launching its Mini App both from an inline button and from the menu button.

Launching the main Mini App

> **TL;DR:** If your bot is a mini app, you can add a prominent **Launch app** button as well as high-quality demo videos and screenshots to the bot's profile. To do this, go to @BotFather and set up your bot's **Main Mini App**.

If your bot is a mini app, you can unlock a number of features that streamline and simplify the way in which users view and interact with it. To do this, go to @BotFather and set up your bot's **Main Mini App**.

After setting a main mini app, you'll be able to upload detailed **media preview demos** to publicly highlight your app's key features on its profile. A **Launch app** button will also appear, allowing users to open your app directly from its profile. Bots that enabled a main mini app will be displayed in the *Apps* tab of the search for users who have launched them.

> Media previews support multiple languages – so you can upload **translated versions** of your previews that will be shown to users based on their **app language**.

A bot's **main Mini App** can also be opened in the current chat by direct link in the format `https://t.me/botusername?startapp`. If a non-empty *startapp* parameter is included in the link, it will be passed to the Mini App in the *start_param* field and in the GET parameter *tgWebAppStartParam*.

Examples

`https://t.me/botusername?startapp`
`https://t.me/botusername?startapp=command`
`https://t.me/botusername?startapp=command&mode=compact`

In this mode, Mini Apps can use the *chat_type* and *chat_instance* parameters to keep track of the current chat context. This introduces support for **concurrent** and **shared** usage by multiple chat members – to create live whiteboards, group orders, multiplayer games and similar apps.

By default, the main Mini App opens to full-screen height, and users cannot reduce them to half-height. However, you can change this behavior via @BotFather or by including the parameter `mode=compact` in

the link to the Mini App, in which case it will open to half-screen height by default.

Good for:

- Fully-fledged web services and integrations that any user can open in one tap.
- Cooperative, multiplayer or teamwork-oriented services within a chat context.
- The use cases are effectively **unlimited**.

> Successful bots which **enable** a main Mini App and **accept payments** in Telegram Stars may be featured in the Telegram Mini App Store. To increase the chances of being featured, we recommend uploading high-quality media showcasing your app on your bot's profile and following our design guidelines.

Inline Mode Mini Apps

> **TL;DR:** Mini Apps launched via **web_app** type InlineQueryResultsButton can be used anywhere in inline mode. Users can create content in a web interface and then seamlessly send it to the current chat via inline mode.

You can use the *button* parameter in the answerInlineQuery method to display a special 'Switch to Mini App' button either above or in place of the inline results. This button will **open a Mini App** from the specified URL. Once done, you can call the Telegram.WebApp.switchInlineQuery method to send the user back to inline mode.

Inline Mini Apps have **no access** to the chat – they can't read messages or send new ones on behalf of the user. To send messages, the user must be redirected to **inline mode** and actively pick a result.

Good for:

- Fully-fledged web services and integrations in inline mode.

Direct Link Mini Apps

> **TL;DR:** Mini App Bots can be launched from a direct link in any chat. They support a *startapp* parameter and are aware of the current chat context.

You can use direct links to **open a Mini App** directly in the current chat. If a non-empty *startapp* parameter is included in the link, it will be passed to the Mini App in the *start_param* field and in the GET parameter *tgWebAppStartParam*.

In this mode, Mini Apps can use the *chat_type* and *chat_instance* parameters to keep track of the current chat context. This introduces support for **concurrent** and **shared** usage by multiple chat members – to create live whiteboards, group orders, multiplayer games and similar apps.

Mini Apps opened from a direct link have **no access** to the chat – they can't read messages or send new ones on behalf of the user. To send messages, the user must be redirected to **inline mode** and actively pick a result.

Starting from Bot API 7.6, by default, Mini Apps of this type open to full-screen height, and users cannot reduce them to half-height. However, you can change this behavior by including the parameter `mode=compact` in the link to the Mini App, in which case it will open to half-screen height by default.

Examples

`https://t.me/botusername/appname`
`https://t.me/botusername/appname?startapp=command`
`https://t.me/botusername/appname?startapp=command&mode=compact`

Good for:

- Fully-fledged web services and integrations that any user can open in one tap.
- Cooperative, multiplayer or teamwork-oriented services within a chat context.
- The use cases are effectively **unlimited**.

Launching Mini Apps on chat join requests

> **TL;DR:** Mini Apps can be launched when the bot is set up as a guard bot of a chat and a user sends join request to the chat. This offers a way to handle join requests using a Mini App.

When a user joins a chat in which the bot was set up as a guard bot, the bot receives an Update "chat_join_request" with the field *query_id*. The bot may call the method sendChatJoinRequestWebApp to show the Mini App to the user before deciding the outcome of the request.

In addition to the user's theme settings, the bot will receive basic user information (`ID` , `name` , `username` , `language_code` , `photo`), the chat info (`ID` , `type` , `title` , `username` , `photo`) and a unique identifier for the join request query **chat_join_request_query_id**, which allows to process the query.

The bot can call the Bot API method answerChatJoinRequestQuery to complete processing of the join request.

Launching Mini Apps from the Attachment Menu

> **TL;DR:** Mini App Bots can request to be added directly to a user's attachment menu, allowing them to be quickly launched from any chat. To try this mode, open this attachment menu link for *@DurgerKingBot*, then use the  menu in **any type of chat**.

Mini App Bots can request to be added directly to a user's attachment menu, allowing them to be quickly launched from **any type of chat**. You can configure in which types of chats your mini app can be started from the attachment menu (private, groups, supergroups or channels).

Attachment menu integration is currently only available for major advertisers on the Telegram Ad Platform. However, **all bots** can use it in the test server environment.

To enable this feature for your bot, open @BotFather from an account on the test server and send the `/setattach` command – or go to *Bot Settings* > *Configure Attachment Menu*. Then specify the URL that will be opened to launch the bot's Mini App via its icon in the attachment menu.

You can add a 'Settings' item to the context menu of your Mini App using @BotFather. When users select this option from the menu, your bot will receive a `settingsButtonClicked` event.

In addition to the user's theme settings, the bot will receive basic user information (`ID` , `name` , `username` , `language_code` , `photo`), as well as public info about the chat partner (`ID` , `name` , `username` , `photo`) or the chat info (`ID` , `type` , `title` , `username` , `photo`) and a unique identifier for the web view session **query_id**, which allows messages of any type to be sent to the chat on behalf of the user that opened the bot.

The bot can call the Bot API method answerWebAppQuery, which sends an inline message from the user via the bot to the chat where it was launched and closes the Mini App.

> You can read more about adding bots to the attachment menu here.

Initializing Mini Apps

To connect your Mini App to the Telegram client, place the script telegram-web-app.js in the `<head>` tag before any other scripts, using this code:

```
<script src="https://telegram.org/js/telegram-web-app.js?64"></script>
```

Once the script is connected, a `window.Telegram.WebApp` object will become available with the following fields:

Field	Type	Description
initData	String	A string with raw data transferred to the Mini App, convenient for validating data. **WARNING:** Validate data from this field before using it on the bot's server.
initDataUnsafe	WebAppInitData	An object with input data transferred to the Mini App. **WARNING:** Data from this field should not be trusted. You should only use data from *initData* on the bot's server and only after it has been validated.
version	String	The version of the Bot API available in the user's Telegram app.
platform	String	The name of the platform of the user's Telegram app.

Field	Type	Description
colorScheme	String	The color scheme currently used in the Telegram app. Either “light” or “dark”. Also available as the CSS variable `var(--tg-color-scheme)`.
themeParams	ThemeParams	An object containing the current theme settings used in the Telegram app.
isActive	Boolean	Bot API 8.0+ *True*, if the Mini App is currently active. *False*, if the Mini App is minimized.
isExpanded	Boolean	*True*, if the Mini App is expanded to the maximum available height. False, if the Mini App occupies part of the screen and can be expanded to the full height using the **expand()** method.
viewportHeight	Float	The current height of the visible area of the Mini App. Also available in CSS as the variable `var(--tg-viewport-height)`. The application can display just the top part of the Mini App, with its lower part remaining outside the screen area. From this position, the user can “pull” the Mini App to its maximum height, while the bot can do the same by calling the **expand()** method. As the position of the Mini App changes, the current height value of the visible area will be updated in real time. Please note that the refresh rate of this value is not sufficient to smoothly follow the lower border of the window. It should not be used to pin interface elements to the bottom of the visible area. It's more appropriate to use the value of the `viewportStableHeight` field for this purpose.

Field	Type	Description
viewportStableHeight	Float	The height of the visible area of the Mini App in its last stable state. Also available in CSS as a variable `var(--tg-viewport-stable-height)`. The application can display just the top part of the Mini App, with its lower part remaining outside the screen area. From this position, the user can “pull” the Mini App to its maximum height, while the bot can do the same by calling the **expand()** method. Unlike the value of `viewportHeight`, the value of `viewportStableHeight` does not change as the position of the Mini App changes with user gestures or during animations. The value of `viewportStableHeight` will be updated after all gestures and animations are completed and the Mini App reaches its final size. *Note the event `viewportChanged` with the passed parameter `isStateStable=true`, which will allow you to track when the stable state of the height of the visible area changes.*
headerColor	String	Current header color in the `#RRGGBB` format.
backgroundColor	String	Current background color in the `#RRGGBB` format.
bottomBarColor	String	Current bottom bar color in the `#RRGGBB` format.
isClosingConfirmationEnabled	Boolean	*True*, if the confirmation dialog is enabled while the user is trying to close the Mini App. *False*, if the confirmation dialog is disabled.
isVerticalSwipesEnabled	Boolean	*True*, if vertical swipes to close or minimize the Mini App are enabled. *False*, if vertical swipes to close or minimize the Mini App are disabled. In any case, the user will still be able to minimize and close the Mini App by swiping the Mini App's header.
isFullscreen	Boolean	*True*, if the Mini App is currently being displayed in fullscreen mode.
isOrientationLocked	Boolean	*True*, if the Mini App’s orientation is currently locked. *False*, if orientation changes freely based on the device’s rotation.
safeAreaInset	SafeAreaInset	An object representing the device's safe area insets, accounting for system UI elements like notches or navigation bars.
contentSafeAreaInset	ContentSafeAreaInset	An object representing the safe area for displaying content within the app, free from overlapping Telegram UI elements.
BackButton	BackButton	An object for controlling the back button which can be displayed in the header of the Mini App in the Telegram interface.

Field	Type	Description
MainButton	BottomButton	An object for controlling the main button, which is displayed at the bottom of the Mini App in the Telegram interface.
SecondaryButton	BottomButton	An object for controlling the secondary button, which is displayed at the bottom of the Mini App in the Telegram interface.
SettingsButton	SettingsButton	An object for controlling the Settings item in the context menu of the Mini App in the Telegram interface.
HapticFeedback	HapticFeedback	An object for controlling haptic feedback.
CloudStorage	CloudStorage	An object for controlling cloud storage.
BiometricManager	BiometricManager	An object for controlling biometrics on the device.
Accelerometer	Accelerometer	An object for accessing accelerometer data on the device.
DeviceOrientation	DeviceOrientation	An object for accessing device orientation data on the device.
Gyroscope	Gyroscope	An object for accessing gyroscope data on the device.
LocationManager	LocationManager	An object for controlling location on the device.
DeviceStorage	DeviceStorage	An object for storing and retrieving data from the device's local storage.
SecureStorage	SecureStorage	An object for storing and retrieving data from the device's secure storage.
Serverless NEW	Serverless	An object for calling the endpoints of the bot's Serverless project.
isVersionAtLeast(version)	Function	Returns true if the user's app supports a version of the Bot API that is equal to or higher than the version passed as the parameter.
setHeaderColor(color)	Function	Bot API 6.1+ A method that sets the app header color in the `#RRGGBB` format. You can also use keywords *bg_color* and *secondary_bg_color*. Up to Bot API 6.9 You can only pass *Telegram.WebApp.themeParams.bg_color* or *Telegram.WebApp.themeParams.secondary_bg_color* as a color or *bg_color*, *secondary_bg_color* keywords.
setBackgroundColor(color)	Function	Bot API 6.1+ A method that sets the app background color in the `#RRGGBB` format. You can also use keywords *bg_color* and *secondary_bg_color*.

Field	Type	Description
setBottomBarColor(color)	Function	Bot API 7.10+ A method that sets the app's bottom bar color in the `#RRGGBB` format. You can also use the keywords *bg_color*, *secondary_bg_color*, and *bottom_bar_bg_color*. This color is also applied to the navigation bar on Android.
enableClosingConfirmation()	Function	Bot API 6.2+ A method that enables a confirmation dialog while the user is trying to close the Mini App.
disableClosingConfirmation()	Function	Bot API 6.2+ A method that disables the confirmation dialog while the user is trying to close the Mini App.
enableVerticalSwipes()	Function	Bot API 7.7+ A method that enables vertical swipes to close or minimize the Mini App. For user convenience, it is recommended to always enable swipes unless they conflict with the Mini App's own gestures.
disableVerticalSwipes()	Function	Bot API 7.7+ A method that disables vertical swipes to close or minimize the Mini App. This method is useful if your Mini App uses swipe gestures that may conflict with the gestures for minimizing and closing the app.
requestFullscreen()	Function	Bot API 8.0+ A method that requests opening the Mini App in fullscreen mode. Although the header is transparent in fullscreen mode, it is recommended that the Mini App sets the header color using the *setHeaderColor* method. This color helps determine a contrasting color for the status bar and other UI controls.
exitFullscreen()	Function	Bot API 8.0+ A method that requests exiting fullscreen mode.
lockOrientation()	Function	Bot API 8.0+ A method that locks the Mini App’s orientation to its current mode (either portrait or landscape). Once locked, the orientation remains fixed, regardless of device rotation. This is useful if a stable orientation is needed during specific interactions.
unlockOrientation()	Function	Bot API 8.0+ A method that unlocks the Mini App’s orientation, allowing it to follow the device's rotation freely. Use this to restore automatic orientation adjustments based on the device orientation.
addToHomeScreen()	Function	Bot API 8.0+ A method that prompts the user to add the Mini App to the home screen. After successfully adding the icon, the `homeScreenAdded` event will be triggered if supported by the device. Note that if the device cannot determine the installation status, the event may not be received even if the icon has been added.

Field	Type	Description
checkHomeScreenStatus([callback])	Function	Bot API 8.0+ A method that checks if adding to the home screen is supported and if the Mini App has already been added. If an optional *callback* parameter is provided, the *callback* function will be called with a single argument *status*, which is a string indicating the home screen status. Possible values for *status* are: - **unsupported** – the feature is not supported, and it is not possible to add the icon to the home screen, - **unknown** – the feature is supported, and the icon can be added, but it is not possible to determine if the icon has already been added, - **added** – the icon has already been added to the home screen, - **missed** – the icon has not been added to the home screen.
onEvent(eventType, eventHandler)	Function	A method that sets the app event handler. Check the list of available events.
offEvent(eventType, eventHandler)	Function	A method that deletes a previously set event handler.
sendData(data)	Function	A method used to send data to the bot. When this method is called, a service message is sent to the bot containing the data *data* of the length up to 4096 bytes, and the Mini App is closed. See the field *web_app_data* in the class Message. *This method is only available for Mini Apps launched via a Keyboard button.*
switchInlineQuery(query[, choose_chat_types])	Function	Bot API 6.7+ A method that inserts the bot's username and the specified inline *query* in the current chat's input field. Query may be empty, in which case only the bot's username will be inserted. If an optional *choose_chat_types* parameter was passed, the client prompts the user to choose a specific chat, then opens that chat and inserts the bot's username and the specified inline query in the input field. You can specify which types of chats the user will be able to choose from. It can be one or more of the following types: *users*, *bots*, *groups*, *channels*.
openLink(url[, options])	Function	A method that opens a link in an external browser. The Mini App will *not* be closed. Bot API 6.4+ If the optional *options* parameter is passed with the field *try_instant_view=true*, the link will be opened in Instant View mode if possible. *Note that this method can be called only in response to user interaction with the Mini App interface (e.g. a click inside the Mini App or on the main button)*

Field	Type	Description
openTelegramLink(url)	Function	A method that opens a telegram link inside the Telegram app. The Mini App will *not* be closed after this method is called. Up to Bot API 7.0 The Mini App *will* be closed after this method is called.
openInvoice(url[, callback])	Function	Bot API 6.1+ A method that opens an invoice using the link *url*. The Mini App will receive the event *invoiceClosed* when the invoice is closed. If an optional *callback* parameter was passed, the *callback* function will be called and the invoice status will be passed as the first argument.
shareToStory(media_url[, params])	Function	Bot API 7.8+ A method that opens the native story editor with the media specified in the *media_url* parameter as an HTTPS URL. An optional *params* argument of the type StoryShareParams describes additional sharing settings.
shareMessage(msg_id[, callback])	Function	Bot API 8.0+ A method that opens a dialog allowing the user to share a message provided by the bot. If an optional *callback* parameter is provided, the *callback* function will be called with a boolean as the first argument, indicating whether the message was successfully sent. The message id passed to this method must belong to a PreparedInlineMessage previously obtained via the Bot API method savePreparedInlineMessage.
setEmojiStatus(custom_emoji_id[, params, callback])	Function	Bot API 8.0+ A method that opens a dialog allowing the user to set the specified custom emoji as their status. An optional *params* argument of type EmojiStatusParams specifies additional settings, such as duration. If an optional *callback* parameter is provided, the *callback* function will be called with a boolean as the first argument, indicating whether the status was set. *Note: this method opens a native dialog and cannot be used to set the emoji status without manual user interaction. For fully programmatic changes, you should instead use the Bot API method setUserEmojiStatus after obtaining authorization to do so via the Mini App method requestEmojiStatusAccess.*
requestEmojiStatusAccess([callback])	Function	Bot API 8.0+ A method that shows a native popup requesting permission for the bot to manage user's emoji status. If an optional *callback* parameter was passed, the *callback* function will be called when the popup is closed and the first argument will be a boolean indicating whether the user granted this access.

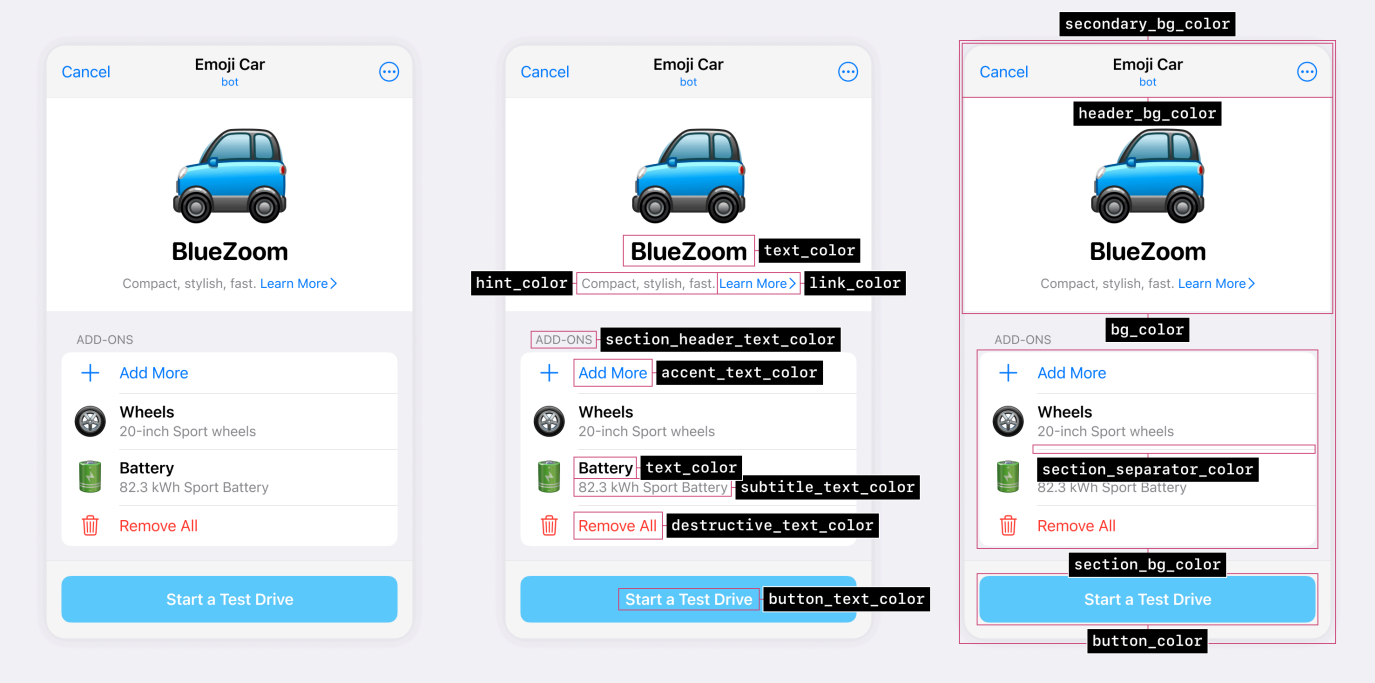
Field	Type	Description
downloadFile(params[, callback])	Function	Bot API 8.0+ A method that displays a native popup prompting the user to download a file specified by the *params* argument of type DownloadFileParams. If an optional *callback* parameter is provided, the *callback* function will be called when the popup is closed, with the first argument as a boolean indicating whether the user accepted the download request.
hideKeyboard()	Function	Bot API 9.1+ A method that hides the on-screen keyboard, if it is currently visible. Does nothing if the keyboard is not active.
showPopup(params[, callback])	Function	Bot API 6.2+ A method that shows a native popup described by the *params* argument of the type PopupParams. The Mini App will receive the event *popupClosed* when the popup is closed. If an optional *callback* parameter was passed, the *callback* function will be called and the field *id* of the pressed button will be passed as the first argument.
showAlert(message[, callback])	Function	Bot API 6.2+ A method that shows *message* in a simple alert with a 'Close' button. If an optional *callback* parameter was passed, the *callback* function will be called when the popup is closed.
showConfirm(message[, callback])	Function	Bot API 6.2+ A method that shows *message* in a simple confirmation window with 'OK' and 'Cancel' buttons. If an optional *callback* parameter was passed, the *callback* function will be called when the popup is closed and the first argument will be a boolean indicating whether the user pressed the 'OK' button.
showScanQrPopup(params[, callback])	Function	Bot API 6.4+ A method that shows a native popup for scanning a QR code described by the *params* argument of the type ScanQrPopupParams. The Mini App will receive the event *qrTextReceived* every time the scanner catches a code with text data. If an optional *callback* parameter was passed, the *callback* function will be called and the text from the QR code will be passed as the first argument. Returning *true* inside this callback function causes the popup to be closed. Starting from Bot API 7.7, the Mini App will receive the *scanQrPopupClosed* event if the user closes the native popup for scanning a QR code.
closeScanQrPopup()	Function	Bot API 6.4+ A method that closes the native popup for scanning a QR code opened with the *showScanQrPopup* method. Run it if you received valid data in the event *qrTextReceived*.

Field	Type	Description
readTextFromClipboard([callback])	Function	Bot API 6.4+ A method that requests text from the clipboard. The Mini App will receive the event *clipboardTextReceived*. If an optional *callback* parameter was passed, the *callback* function will be called and the text from the clipboard will be passed as the first argument. *Note: this method can be called only for Mini Apps launched from the attachment menu and only in response to a user interaction with the Mini App interface (e.g. a click inside the Mini App or on the main button).*
requestWriteAccess([callback])	Function	Bot API 6.9+ A method that shows a native popup requesting permission for the bot to send messages to the user. If an optional *callback* parameter was passed, the *callback* function will be called when the popup is closed and the first argument will be a boolean indicating whether the user granted this access.
requestContact([callback])	Function	Bot API 6.9+ A method that shows a native popup prompting the user for their phone number. If an optional *callback* parameter was passed, the *callback* function will be called when the popup is closed and the first argument will be a boolean indicating whether the user shared its phone number.
requestChat(req_id[, callback])	Function	Bot API 9.6+ A method that opens a dialog allowing the user to select an existing chat or create a new one. If an optional *callback* parameter is provided, the *callback* function will be called with a boolean as the first argument, indicating whether the message was successfully sent. The request id passed to this method must belong to a PreparedKeyboardButton previously obtained via the Bot API method savePreparedKeyboardButton.
ready()	Function	A method that informs the Telegram app that the Mini App is ready to be displayed. It is recommended to call this method as early as possible, as soon as all essential interface elements are loaded. Once this method is called, the loading placeholder is hidden and the Mini App is shown. If the method is not called, the placeholder will be hidden only when the page is fully loaded.
expand()	Function	A method that expands the Mini App to the maximum available height. To find out if the Mini App is expanded to the maximum height, refer to the value of the *Telegram.WebApp.isExpanded* parameter
close()	Function	A method that closes the Mini App.

ThemeParams

Mini Apps can adjust the appearance of the interface to match the Telegram user's app in real time. This object contains the user's current theme settings:

Field	Type	Description
bg_color	String	*Optional*. Background color in the `#RRGGBB` format. Also available as the CSS variable `var(--tg-theme-bg-color)`.
text_color	String	*Optional*. Main text color in the `#RRGGBB` format. Also available as the CSS variable `var(--tg-theme-text-color)`.
hint_color	String	*Optional*. Hint text color in the `#RRGGBB` format. Also available as the CSS variable `var(--tg-theme-hint-color)`.
link_color	String	*Optional*. Link color in the `#RRGGBB` format. Also available as the CSS variable `var(--tg-theme-link-color)`.
button_color	String	*Optional*. Button color in the `#RRGGBB` format. Also available as the CSS variable `var(--tg-theme-button-color)`.
button_text_color	String	*Optional*. Button text color in the `#RRGGBB` format. Also available as the CSS variable `var(--tg-theme-button-text-color)`.
secondary_bg_color	String	*Optional*. Bot API 6.1+ Secondary background color in the `#RRGGBB` format. Also available as the CSS variable `var(--tg-theme-secondary-bg-color)`.
header_bg_color	String	*Optional*. Bot API 7.0+ Header background color in the `#RRGGBB` format. Also available as the CSS variable `var(--tg-theme-header-bg-color)`.
bottom_bar_bg_color	String	*Optional*. Bot API 7.10+ Bottom background color in the `#RRGGBB` format. Also available as the CSS variable `var(--tg-theme-bottom-bar-bg-color)`.
accent_text_color	String	*Optional*. Bot API 7.0+ Accent text color in the `#RRGGBB` format. Also available as the CSS variable `var(--tg-theme-accent-text-color)`.
section_bg_color	String	*Optional*. Bot API 7.0+ Background color for the section in the `#RRGGBB` format. It is recommended to use this in conjunction with *secondary_bg_color*. Also available as the CSS variable `var(--tg-theme-section-bg-color)`.
section_header_text_color	String	*Optional*. Bot API 7.0+ Header text color for the section in the `#RRGGBB` format. Also available as the CSS variable `var(--tg-theme-section-header-text-color)`.
section_separator_color	String	*Optional*. Bot API 7.6+ Section separator color in the `#RRGGBB` format. Also available as the CSS variable `var(--tg-theme-section-separator-color)`.
subtitle_text_color	String	*Optional*. Bot API 7.0+ Subtitle text color in the `#RRGGBB` format. Also available as the CSS variable `var(--tg-theme-subtitle-text-color)`.
destructive_text_color	String	*Optional*. Bot API 7.0+ Text color for destructive actions in the `#RRGGBB` format. Also available as the CSS variable `var(--tg-theme-destructive-text-color)`.



StoryShareParams

This object describes additional sharing settings for the native story editor.

Field	Type	Description
text	String	*Optional*. The caption to be added to the media, 0-200 characters for regular users and 0-2048 characters for premium subscribers.
widget_link	StoryWidgetLink	*Optional*. An object that describes a widget link to be included in the story. Note that only premium subscribers can post stories with links.

StoryWidgetLink

This object describes a widget link to be included in the story.

Field	Type	Description
url	String	The URL to be included in the story.
name	String	*Optional*. The name to be displayed for the widget link, 0-48 characters.

ScanQrPopupParams

This object describes the native popup for scanning QR codes.

Field	Type	Description
text	String	*Optional*. The text to be displayed under the 'Scan QR' heading, 0-64 characters.

PopupParams

This object describes the native popup.

Field	Type	Description
title	String	*Optional*. The text to be displayed in the popup title, 0-64 characters.
message	String	The message to be displayed in the body of the popup, 1-256 characters.
buttons	Array of PopupButton	*Optional*. List of buttons to be displayed in the popup, 1-3 buttons. Set to *[{“type”:“close”}]* by default.

PopupButton

This object describes the native popup button.

Field	Type	Description
id	String	*Optional*. Identifier of the button, 0-64 characters. Set to empty string by default. If the button is pressed, its *id* is returned in the callback and the *popupClosed* event.
type	String	*Optional*. Type of the button. Set to *default* by default. Can be one of these values: - *default*, a button with the default style, - *ok*, a button with the localized text “OK”, - *close*, a button with the localized text “Close”, - *cancel*, a button with the localized text “Cancel”, - *destructive*, a button with a style that indicates a destructive action (e.g. “Remove”, “Delete”, etc.).
text	String	*Optional*. The text to be displayed on the button, 0-64 characters. Required if *type* is *default* or *destructive*. Irrelevant for other types.

EmojiStatusParams

This object describes additional settings for setting an emoji status.

Field	Type	Description
duration	Integer	*Optional*. The duration for which the status will remain set, in seconds.

DownloadFileParams

This object describes the parameters for the file download request.

> **Note:** To ensure consistent file download behavior across platforms, include the HTTP headers `Content-Disposition: attachment; filename="<file_name>"` and `Access-Control-Allow-Origin: https://web.telegram.org` in the server response. Without these headers, the download may not work as expected, especially on web platforms.

Field	Type	Description
url	String	The HTTPS URL of the file to be downloaded.
file_name	String	The suggested name for the downloaded file.

SafeAreaInset

This object represents the system-defined safe area insets, providing padding values to ensure content remains within visible boundaries, avoiding overlap with system UI elements like notches or navigation bars.

Field	Type	Description
top	Integer	The top inset in pixels, representing the space to avoid at the top of the screen. Also available as the CSS variable `var(--tg-safe-area-inset-top)`.
bottom	Integer	The bottom inset in pixels, representing the space to avoid at the bottom of the screen. Also available as the CSS variable `var(--tg-safe-area-inset-bottom)`.
left	Integer	The left inset in pixels, representing the space to avoid on the left side of the screen. Also available as the CSS variable `var(--tg-safe-area-inset-left)`.
right	Integer	The right inset in pixels, representing the space to avoid on the right side of the screen. Also available as the CSS variable `var(--tg-safe-area-inset-right)`.



ContentSafeAreaInset

This object represents the content-defined safe area insets, providing padding values to ensure content remains within visible boundaries, avoiding overlap with Telegram UI elements.

Field	Type	Description
top	Integer	The top inset in pixels, representing the space to avoid at the top of the content area. Also available as the CSS variable `var(--tg-content-safe-area-inset-top)`.
bottom	Integer	The bottom inset in pixels, representing the space to avoid at the bottom of the content area. Also available as the CSS variable `var(--tg-content-safe-area-inset-bottom)`.
left	Integer	The left inset in pixels, representing the space to avoid on the left side of the content area. Also available as the CSS variable `var(--tg-content-safe-area-inset-left)`.

Field	Type	Description
right	Integer	The right inset in pixels, representing the space to avoid on the right side of the content area. Also available as the CSS variable `var(--tg-content-safe-area-inset-right)`.



BackButton

This object controls the **back** button, which can be displayed in the header of the Mini App in the Telegram interface.

Field	Type	Description
isVisible	Boolean	Shows whether the button is visible. Set to *false* by default.
onClick(callback)	Function	Bot API 6.1+ A method that sets the button press event handler. An alias for `Telegram.WebApp.onEvent('backButtonClicked', callback)`
offClick(callback)	Function	Bot API 6.1+ A method that removes the button press event handler. An alias for `Telegram.WebApp.offEvent('backButtonClicked', callback)`
show()	Function	Bot API 6.1+ A method to make the button active and visible.
hide()	Function	Bot API 6.1+ A method to hide the button.

All these methods return the BackButton object so they can be chained.

BottomButton

This object controls the button that is displayed at the bottom of the Mini App in the Telegram interface.

Field	Type	Description
type	String	*Readonly.* Type of the button. It can be either *main* for the main button or *secondary* for the secondary button.

Field	Type	Description
iconCustomEmojiId	String	Bot API 9.5+ Unique identifier of the custom emoji shown before the text of the button.
text	String	Current button text. Set to *Continue* for the main button and *Cancel* for the secondary button by default.
color	String	Current button color. Set to *themeParams.button_color* for the main button and *themeParams.bottom_bar_bg_color* for the secondary button by default.
textColor	String	Current button text color. Set to *themeParams.button_text_color* for the main button and *themeParams.button_color* for the secondary button by default.
isVisible	Boolean	Shows whether the button is visible. Set to *false* by default.
isActive	Boolean	Shows whether the button is active. Set to *true* by default.
hasShineEffect	Boolean	Bot API 7.10+ Shows whether the button has a shine effect. Set to *false* by default.
position	String	Bot API 7.10+ Position of the secondary button. Not defined for the main button. It applies only if both the main and secondary buttons are visible. Set to *left* by default. Supported values: - *left*, displayed to the left of the main button, - *right*, displayed to the right of the main button, - *top*, displayed above the main button, - *bottom*, displayed below the main button.
isProgressVisible	Boolean	*Readonly.* Shows whether the button is displaying a loading indicator.
setText(text)	Function	A method to set the button text.
onClick(callback)	Function	A method that sets the button's press event handler. An alias for `Telegram.WebApp.onEvent('mainButtonClicked', callback)`
offClick(callback)	Function	A method that removes the button's press event handler. An alias for `Telegram.WebApp.offEvent('mainButtonClicked', callback)`
show()	Function	A method to make the button visible. *Note that opening the Mini App from the attachment menu hides the main button until the user interacts with the Mini App interface.*
hide()	Function	A method to hide the button.
enable()	Function	A method to enable the button.
disable()	Function	A method to disable the button.
showProgress(leaveActive)	Function	A method to show a loading indicator on the button. It is recommended to display loading progress if the action tied to the button may take a long time. By default, the button is disabled while the action is in progress. If the parameter `leaveActive=true` is passed, the button remains enabled.
hideProgress()	Function	A method to hide the loading indicator.

Field	Type	Description
setParams(params)	Function	A method to set the button parameters. The *params* parameter is an object containing one or several fields that need to be changed: **icon_custom_emoji_id** - Bot API 9.5+ button icon emoji id; **text** - button text; **color** - button color; **text_color** - button text color; **has_shine_effect** - Bot API 7.10+ enable shine effect; **position** - position of the secondary button; **is_active** - enable the button; **is_visible** - show the button.

All these methods return the BottomButton object so they can be chained.

SettingsButton

This object controls the **Settings** item in the context menu of the Mini App in the Telegram interface.

Field	Type	Description
isVisible	Boolean	Shows whether the context menu item is visible. Set to *false* by default.
onClick(callback)	Function	Bot API 7.0+ A method that sets the press event handler for the Settings item in the context menu. An alias for `Telegram.WebApp.onEvent('settingsButtonClicked', callback)`
offClick(callback)	Function	Bot API 7.0+ A method that removes the press event handler from the Settings item in the context menu. An alias for `Telegram.WebApp.offEvent('settingsButtonClicked', callback)`
show()	Function	Bot API 7.0+ A method to make the Settings item in the context menu visible.
hide()	Function	Bot API 7.0+ A method to hide the Settings item in the context menu.

All these methods return the SettingsButton object so they can be chained.

HapticFeedback

This object controls haptic feedback.

Field	Type	Description
impactOccurred(style)	Function	Bot API 6.1+ A method tells that an impact occurred. The Telegram app may play the appropriate haptics based on style value passed. Style can be one of these values: - *light*, indicates a collision between small or lightweight UI objects, - *medium*, indicates a collision between medium-sized or medium-weight UI objects, - *heavy*, indicates a collision between large or heavyweight UI objects, - *rigid*, indicates a collision between hard or inflexible UI objects, - *soft*, indicates a collision between soft or flexible UI objects.

Field	Type	Description
notificationOccurred(type)	Function	Bot API 6.1+ A method tells that a task or action has succeeded, failed, or produced a warning. The Telegram app may play the appropriate haptics based on type value passed. Type can be one of these values: - *error*, indicates that a task or action has failed, - *success*, indicates that a task or action has completed successfully, - *warning*, indicates that a task or action produced a warning.
selectionChanged()	Function	Bot API 6.1+ A method tells that the user has changed a selection. The Telegram app may play the appropriate haptics. *Do not use this feedback when the user makes or confirms a selection; use it only when the selection changes.*

All these methods return the HapticFeedback object so they can be chained.

CloudStorage

This object controls the cloud storage. Each bot can store up to 1024 items per user in the cloud storage.

Field	Type	Description
setItem(key, value[, callback])	Function	Bot API 6.9+ A method that stores a value in the cloud storage using the specified key. The key should contain 1-128 characters, only `A-Z`, `a-z`, `0-9`, `_` and `-` are allowed. The value should contain 0-4096 characters. You can store up to 1024 keys in the cloud storage. If an optional *callback* parameter was passed, the *callback* function will be called. In case of an error, the first argument will contain the error. In case of success, the first argument will be *null* and the second argument will be a boolean indicating whether the value was stored.
getItem(key, callback)	Function	Bot API 6.9+ A method that receives a value from the cloud storage using the specified key. The key should contain 1-128 characters, only `A-Z`, `a-z`, `0-9`, `_` and `-` are allowed. In case of an error, the *callback* function will be called and the first argument will contain the error. In case of success, the first argument will be *null* and the value will be passed as the second argument.
getItems(keys, callback)	Function	Bot API 6.9+ A method that receives values from the cloud storage using the specified keys. The keys should contain 1-128 characters, only `A-Z`, `a-z`, `0-9`, `_` and `-` are allowed. In case of an error, the *callback* function will be called and the first argument will contain the error. In case of success, the first argument will be *null* and the values will be passed as the second argument.
removeItem(key[, callback])	Function	Bot API 6.9+ A method that removes a value from the cloud storage using the specified key. The key should contain 1-128 characters, only `A-Z`, `a-z`, `0-9`, `_` and `-` are allowed. If an optional *callback* parameter was passed, the *callback* function will be called. In case of an error, the first argument will contain the error. In case of success, the first argument will be *null* and the second argument will be a boolean indicating whether the value was removed.

Field	Type	Description
removeItems(keys[, callback])	Function	Bot API 6.9+ A method that removes values from the cloud storage using the specified keys. The keys should contain 1-128 characters, only `A-Z`, `a-z`, `0-9`, `_` and `-` are allowed. If an optional *callback* parameter was passed, the *callback* function will be called. In case of an error, the first argument will contain the error. In case of success, the first argument will be *null* and the second argument will be a boolean indicating whether the values were removed.
getKeys(callback)	Function	Bot API 6.9+ A method that receives the list of all keys stored in the cloud storage. In case of an error, the *callback* function will be called and the first argument will contain the error. In case of success, the first argument will be *null* and the list of keys will be passed as the second argument.

All these methods return the CloudStorage object, so they can be chained.

BiometricManager

This object controls biometrics on the device. Before the first use of this object, it needs to be initialized using the *init* method.

Field	Type	Description
isInited	Boolean	Shows whether biometrics object is initialized.
isBiometricAvailable	Boolean	Shows whether biometrics is available on the current device.
biometricType	String	The type of biometrics currently available on the device. Can be one of these values: - *finger*, fingerprint-based biometrics, - *face*, face-based biometrics, - *unknown*, biometrics of an unknown type.
isAccessRequested	Boolean	Shows whether permission to use biometrics has been requested.
isAccessGranted	Boolean	Shows whether permission to use biometrics has been granted.
isBiometricTokenSaved	Boolean	Shows whether the token is saved in secure storage on the device.
deviceId	String	A unique device identifier that can be used to match the token to the device.
init([callback])	Function	Bot API 7.2+ A method that initializes the BiometricManager object. It should be called before the object's first use. If an optional *callback* parameter was passed, the *callback* function will be called when the object is initialized.
requestAccess(params[, callback])	Function	Bot API 7.2+ A method that requests permission to use biometrics according to the *params* argument of type BiometricRequestAccessParams. If an optional *callback* parameter was passed, the *callback* function will be called and the first argument will be a boolean indicating whether the user granted access.

Field	Type	Description
authenticate(params[, callback])	Function	Bot API 7.2+ A method that authenticates the user using biometrics according to the *params* argument of type BiometricAuthenticateParams. If an optional *callback* parameter was passed, the *callback* function will be called and the first argument will be a boolean indicating whether the user authenticated successfully. If so, the second argument will be a biometric token.
updateBiometricToken(token, [callback])	Function	Bot API 7.2+ A method that updates the biometric token in secure storage on the device. To remove the token, pass an empty string. If an optional *callback* parameter was passed, the *callback* function will be called and the first argument will be a boolean indicating whether the token was updated.
openSettings()	Function	Bot API 7.2+ A method that opens the biometric access settings for bots. Useful when you need to request biometrics access to users who haven't granted it yet. *Note that this method can be called only in response to user interaction with the Mini App interface (e.g. a click inside the Mini App or on the main button)*

All these methods return the BiometricManager object so they can be chained.

BiometricRequestAccessParams

This object describes the native popup for requesting permission to use biometrics.

Field	Type	Description
reason	String	*Optional*. The text to be displayed to a user in the popup describing why the bot needs access to biometrics, 0-128 characters.

BiometricAuthenticateParams

This object describes the native popup for authenticating the user using biometrics.

Field	Type	Description
reason	String	*Optional*. The text to be displayed to a user in the popup describing why you are asking them to authenticate and what action you will be taking based on that authentication, 0-128 characters.

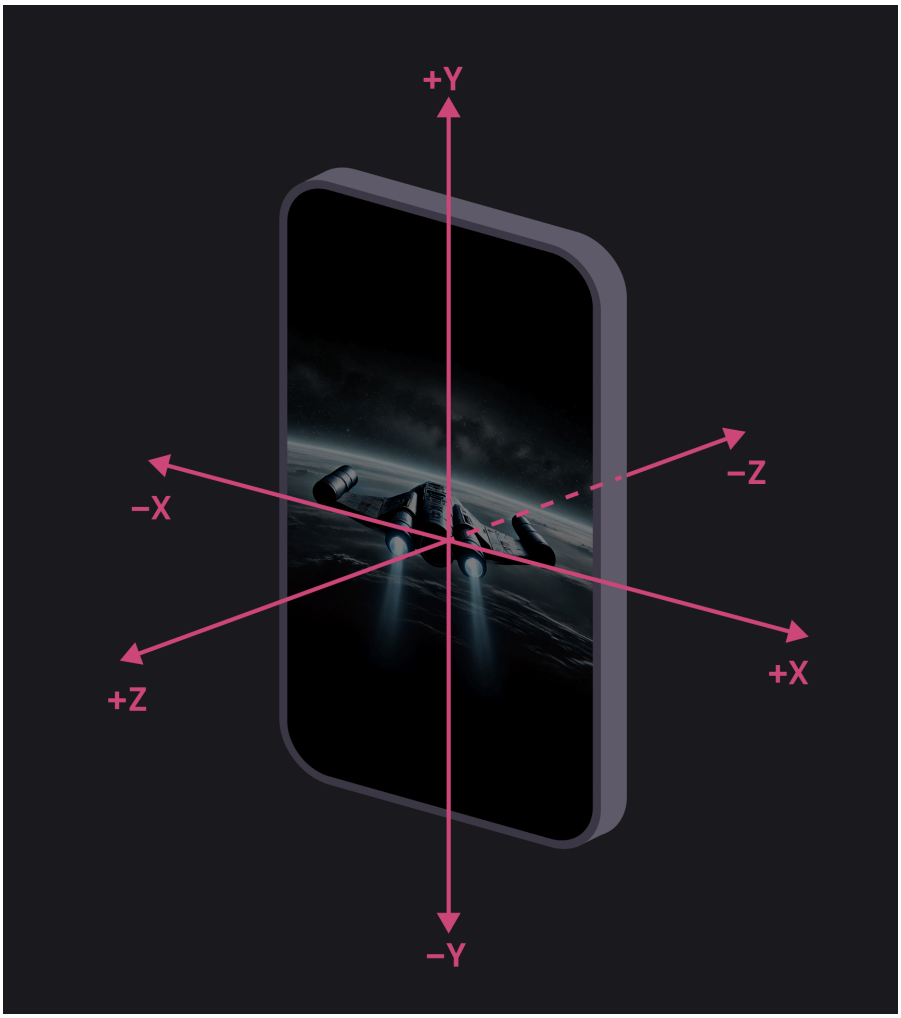
Accelerometer

This object provides access to accelerometer data on the device.

Field	Type	Description
isStarted	Boolean	Indicates whether accelerometer tracking is currently active.
x	Float	The current acceleration in the X-axis, measured in m/s².
y	Float	The current acceleration in the Y-axis, measured in m/s².

Field	Type	Description
z	Float	The current acceleration in the Z-axis, measured in m/s².
start(params[, callback])	Function	Bot API 8.0+ Starts tracking accelerometer data using *params* of type AccelerometerStartParams. If an optional *callback* parameter is provided, the *callback* function will be called with a boolean indicating whether tracking was successfully started.
stop([callback])	Function	Bot API 8.0+ Stops tracking accelerometer data. If an optional *callback* parameter is provided, the *callback* function will be called with a boolean indicating whether tracking was successfully stopped.

All these methods return the Accelerometer object so they can be chained.



AccelerometerStartParams

This object defines the parameters for starting accelerometer tracking.

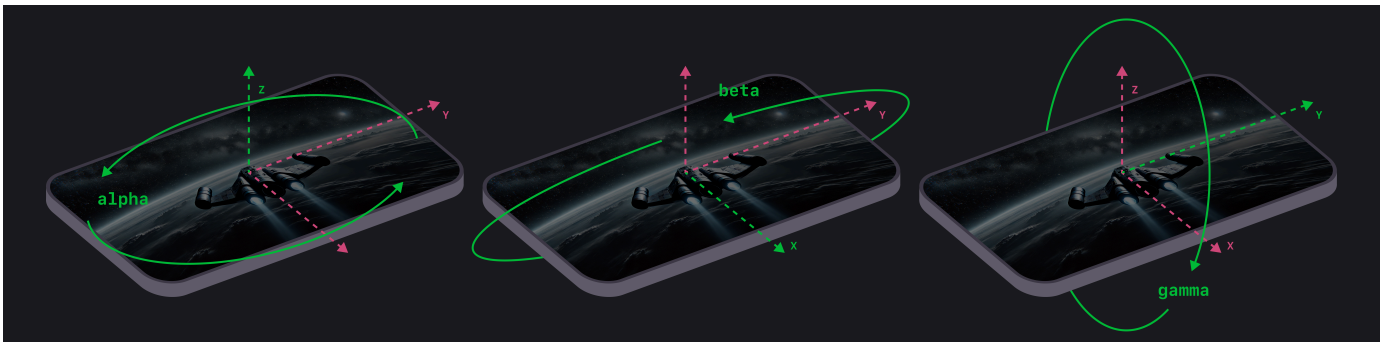
Field	Type	Description
refresh_rate	Integer	*Optional.* The refresh rate in milliseconds, with acceptable values ranging from 20 to 1000. Set to *1000* by default. Note that *refresh_rate* may not be supported on all platforms, so the actual tracking frequency may differ from the specified value.

DeviceOrientation

This object provides access to orientation data on the device.

Field	Type	Description
isStarted	Boolean	Indicates whether device orientation tracking is currently active.
absolute	Boolean	A boolean that indicates whether or not the device is providing orientation data in absolute values.
alpha	Float	The rotation around the Z-axis, measured in radians.
beta	Float	The rotation around the X-axis, measured in radians.
gamma	Float	The rotation around the Y-axis, measured in radians.
start(params[, callback])	Function	Bot API 8.0+ Starts tracking device orientation data using *params* of type DeviceOrientationStartParams. If an optional *callback* parameter is provided, the *callback* function will be called with a boolean indicating whether tracking was successfully started.
stop([callback])	Function	Bot API 8.0+ Stops tracking device orientation data. If an optional *callback* parameter is provided, the *callback* function will be called with a boolean indicating whether tracking was successfully stopped.

All these methods return the DeviceOrientation object so they can be chained.



DeviceOrientationStartParams

This object defines the parameters for starting device orientation tracking.

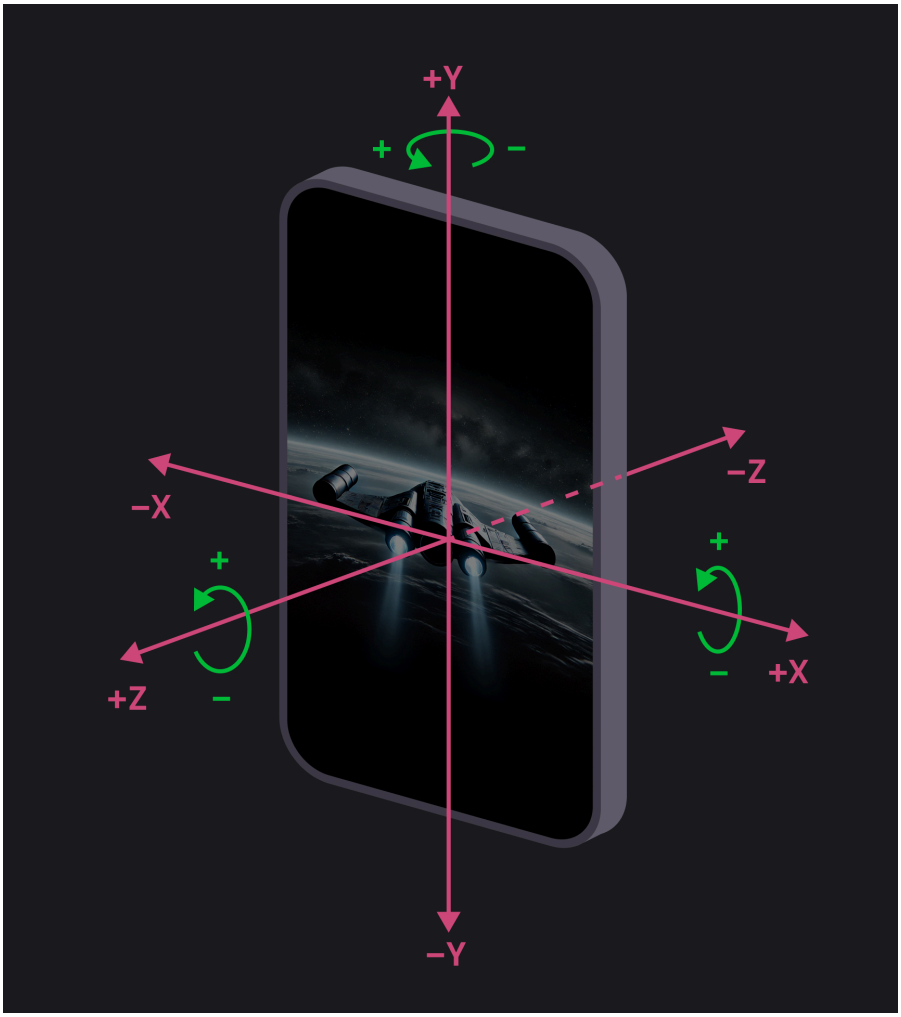
Field	Type	Description
refresh_rate	Integer	*Optional.* The refresh rate in milliseconds, with acceptable values ranging from 20 to 1000. Set to *1000* by default. Note that *refresh_rate* may not be supported on all platforms, so the actual tracking frequency may differ from the specified value.
need_absolute	Boolean	*Optional.* Pass *true* to receive absolute orientation data, allowing you to determine the device's attitude relative to magnetic north. Use this option if implementing features like a compass in your app. If relative data is sufficient, pass *false*. Set to *false* by default. **Note:** Keep in mind that some devices may not support absolute orientation data. In such cases, you will receive relative data even if *need_absolute=true* is passed. Check the *DeviceOrientation.absolute* parameter to determine whether the data provided is absolute or relative.

Gyroscope

This object provides access to gyroscope data on the device.

Field	Type	Description
isStarted	Boolean	Indicates whether gyroscope tracking is currently active.
x	Float	The current rotation rate around the X-axis, measured in rad/s.
y	Float	The current rotation rate around the Y-axis, measured in rad/s.
z	Float	The current rotation rate around the Z-axis, measured in rad/s.
start(params[, callback])	Function	Bot API 8.0+ Starts tracking gyroscope data using *params* of type GyroscopeStartParams. If an optional *callback* parameter is provided, the *callback* function will be called with a boolean indicating whether tracking was successfully started.
stop([callback])	Function	Bot API 8.0+ Stops tracking gyroscope data. If an optional *callback* parameter is provided, the *callback* function will be called with a boolean indicating whether tracking was successfully stopped.

All these methods return the Gyroscope object so they can be chained.



GyroscopeStartParams

This object defines the parameters for starting gyroscope tracking.

Field	Type	Description
refresh_rate	Integer	*Optional.* The refresh rate in milliseconds, with acceptable values ranging from 20 to 1000. Set to *1000* by default. Note that *refresh_rate* may not be supported on all platforms, so the actual tracking frequency may differ from the specified value.

LocationManager

This object controls location access on the device. Before the first use of this object, it needs to be initialized using the *init* method.

Field	Type	Description
isInited	Boolean	Shows whether the LocationManager object has been initialized.
isLocationAvailable	Boolean	Shows whether location services are available on the current device.
isAccessRequested	Boolean	Shows whether permission to use location has been requested.
isAccessGranted	Boolean	Shows whether permission to use location has been granted.
init([callback])	Function	Bot API 8.0+ A method that initializes the LocationManager object. It should be called before the object's first use. If an optional *callback* parameter is provided, the *callback* function will be called when the object is initialized.
getLocation(callback)	Function	Bot API 8.0+ A method that requests location data. The *callback* function will be called with *null* as the first argument if access to location was not granted, or an object of type LocationData as the first argument if access was successful.
openSettings()	Function	Bot API 8.0+ A method that opens the location access settings for bots. Useful when you need to request location access from users who haven't granted it yet. *Note that this method can be called only in response to user interaction with the Mini App interface (e.g., a click inside the Mini App or on the main button).*

All these methods return the LocationManager object so they can be chained.

LocationData

This object contains data about the current location.

Field	Type	Description
latitude	Float	Latitude in degrees.
longitude	Float	Longitude in degrees.
altitude	Float	Altitude above sea level in meters. *null* if altitude data is not available on the device.
course	Float	The direction the device is moving in degrees (0 = North, 90 = East, 180 = South, 270 = West). *null* if course data is not available on the device.
speed	Float	The speed of the device in m/s. *null* if speed data is not available on the device.

Field	Type	Description
horizontal_accuracy	Float	Accuracy of the latitude and longitude values in meters. *null* if horizontal accuracy data is not available on the device.
vertical_accuracy	Float	Accuracy of the altitude value in meters. *null* if vertical accuracy data is not available on the device.
course_accuracy	Float	Accuracy of the course value in degrees. *null* if course accuracy data is not available on the device.
speed_accuracy	Float	Accuracy of the speed value in m/s. *null* if speed accuracy data is not available on the device.

DeviceStorage

This object provides access to persistent local storage on the user's device. It is conceptually similar to the browser's `localStorage` API, but integrated within the Telegram client. All data is stored locally and is available only to the bot that created it. Each bot can store up to **5 MB per user** using this storage.

Field	Type	Description
setItem(key, value[, callback])	Function	Bot API 9.0+ A method that stores a value in the device's local storage using the specified key. If an optional *callback* parameter was passed, the *callback* function will be called. In case of an error, the first argument will contain the error. In case of success, the first argument will be *null* and the second argument will be a boolean indicating whether the value was stored.
getItem(key, callback)	Function	Bot API 9.0+ A method that receives a value from the device's local storage using the specified key. In case of an error, the *callback* function will be called and the first argument will contain the error. In case of success, the first argument will be *null* and the value will be passed as the second argument.
removeItem(key[, callback])	Function	Bot API 9.0+ A method that removes a value from the device's local storage using the specified key. If an optional *callback* parameter was passed, the *callback* function will be called. In case of an error, the first argument will contain the error. In case of success, the first argument will be *null* and the second argument will be a boolean indicating whether the value was removed.
clear([callback])	Function	Bot API 9.0+ A method that clears all keys previously stored by the bot in the device's local storage. If an optional *callback* parameter was passed, the *callback* function will be called. In case of an error, the first argument will contain the error. In case of success, the first argument will be *null* and the second argument will be a boolean indicating whether all values were removed.

All these methods return the DeviceStorage object, so they can be chained.

SecureStorage

This object provides access to a secure storage on the user's device for sensitive data. On **iOS**, it uses the system **Keychain**; on **Android**, it uses the **Keystore**. This ensures that all stored values are encrypted at rest and inaccessible to unauthorized applications.

Secure storage is suitable for storing tokens, secrets, authentication state, and other sensitive user-specific information. Each bot can store up to **10 items per user**.

Field	Type	Description
setItem(key, value[, callback])	Function	Bot API 9.0+ A method that stores a value in the device's secure storage using the specified key. If an optional *callback* parameter was passed, the *callback* function will be called. In case of an error, the first argument will contain the error. In case of success, the first argument will be *null* and the second argument will be a boolean indicating whether the value was stored.
getItem(key, callback)	Function	Bot API 9.0+ A method that receives a value from the device's secure storage using the specified key. In case of an error, the *callback* function will be called and the first argument will contain the error. In case of success, the first argument will be *null* and the value will be passed as the second argument. If the key was not found, the second argument will be *null*, and the third argument will be a boolean indicating whether the key can be restored from the current device.
restoreItem(key[, callback])	Function	Bot API 9.0+ Attempts to restore a key that previously existed on the current device. When called, the user will be asked for permission to restore the value. If the user declines or an error occurs, the first argument in the *callback* will contain the error. If restored successfully, the first argument will be *null* and the second argument will contain the restored value.
removeItem(key[, callback])	Function	Bot API 9.0+ A method that removes a value from the device's secure storage using the specified key. If an optional *callback* parameter was passed, the *callback* function will be called. In case of an error, the first argument will contain the error. In case of success, the first argument will be *null* and the second argument will be a boolean indicating whether the value was removed.
clear([callback])	Function	Bot API 9.0+ A method that clears all keys previously stored by the bot in the device's secure storage. If an optional *callback* parameter was passed, the *callback* function will be called. In case of an error, the first argument will contain the error. In case of success, the first argument will be *null* and the second argument will be a boolean indicating whether all values were removed.

All these methods return the SecureStorage object, so they can be chained.

Serverless

This object lets a Mini App hosted on Telegram Serverless call the bot's endpoints – the server-side functions of the Mini App. Each call is sent together with the Mini App's *initData*, which the platform validates before running the endpoint, so the endpoint always knows which user is calling it.

Field	Type	Description
call(name[, input][, callback])	Function	A method that calls the endpoint with the specified name. The optional *input* parameter is a JSON-serializable object that is passed to the endpoint as its first argument. If an optional *callback* parameter was passed, the *callback* function will be called. In case of an error, the first argument will contain the error. In case of success, the first argument will be *null* and the value returned by the endpoint will be passed as the second argument.

The error passed to the *callback* function is an *Error* object with the following additional fields:

Field	Type	Description
message	String	A human-readable description of the error.

Field	Type	Description
type	String	*Optional*. The type of the error returned by the platform. “ENDPOINT_ERROR” if the endpoint refused the call by throwing an *EndpointError*. Other values indicate errors reported by the platform, such as “UNAUTHORIZED” if *initData* is missing or invalid.
status	Integer	The HTTP status code of the response, or *0* if the request could not be completed.
parameters	Object	*Optional*. Additional data passed by the endpoint along with its *EndpointError*.

```
Telegram.WebApp.Serverless.call('getProfile', {lang: 'en'}, function(error, profile) {
  if (error) {
    console.error(error.message);
    return;
  }
  console.log(profile);
});
```

WebAppInitData

This object contains data that is transferred to the Mini App when it is opened. It is empty if the Mini App was launched from a keyboard button or from inline mode.

Field	Type	Description
query_id	String	*Optional.* A unique identifier for the Mini App session, required for sending messages via the answerWebAppQuery method.
chat_join_request_query_id	String	*Optional.* A unique identifier for the chat join request query, required for processing the query via the method answerChatJoinRequestQuery.
user	WebAppUser	*Optional.* An object containing data about the current user.
receiver	WebAppUser	*Optional.* An object containing data about the chat partner of the current user in the chat where the bot was launched via the attachment menu. Returned only for private chats and only for Mini Apps launched via the attachment menu.
chat	WebAppChat	*Optional.* An object containing data about the chat where the bot was launched via the attachment menu or the chat to which the join request query was sent. Returned for supergroups, channels and group chats – only for Mini Apps launched via the attachment menu and chat join requests.
chat_type	String	*Optional.* Type of the chat from which the Mini App was opened. Can be either “sender” for a private chat with the user opening the link, “private”, “group”, “supergroup”, or “channel”. Returned only for Mini Apps launched from direct links.
chat_instance	String	*Optional.* Global identifier, uniquely corresponding to the chat from which the Mini App was opened. Returned only for Mini Apps launched from a direct link.

Field	Type	Description
start_param	String	*Optional.* The value of the *startattach* parameter, passed via link. Only returned for Mini Apps when launched from the attachment menu via link. The value of the `start_param` parameter will also be passed in the GET-parameter `tgWebAppStartParam`, so the Mini App can load the correct interface right away.
can_send_after	Integer	*Optional.* Time in seconds, after which a message can be sent via the answerWebAppQuery method.
auth_date	Integer	Unix time when the form was opened.
hash	String	A hash of all passed parameters, which the bot server can use to check their validity.
signature	String	A signature of all passed parameters (except *hash*), which the third party can use to check their validity.

WebAppUser

This object contains the data of the Mini App user.

Field	Type	Description
id	Integer	A unique identifier for the user or bot. This number may have more than 32 significant bits and some programming languages may have difficulty/silent defects in interpreting it. It has at most 52 significant bits, so a 64-bit integer or a double-precision float type is safe for storing this identifier.
is_bot	Boolean	*Optional*. *True*, if this user is a bot. Returns in the receiver field only.
first_name	String	First name of the user or bot.
last_name	String	*Optional*. Last name of the user or bot.
username	String	*Optional*. Username of the user or bot.
language_code	String	*Optional*. IETF language tag of the user's language. Returns in *user* field only.
is_premium	True	*Optional*. *True*, if this user is a Telegram Premium user.
added_to_attachment_menu	True	*Optional*. *True*, if this user added the bot to the attachment menu.
allows_write_to_pm	True	*Optional*. *True*, if this user allowed the bot to message them.
photo_url	String	*Optional*. URL of the user’s profile photo. The photo can be in .jpeg or .svg formats.

WebAppChat

This object represents a chat.

Field	Type	Description
id	Integer	Unique identifier for this chat. This number may have more than 32 significant bits and some programming languages may have difficulty/silent defects in interpreting it. But it has at most 52 significant bits, so a signed 64-bit integer or double-precision float type are safe for storing this identifier.
type	String	Type of chat, can be either “group”, “supergroup” or “channel”
title	String	Title of the chat
username	String	*Optional*. Username of the chat
photo_url	String	*Optional*. URL of the chat’s photo. The photo can be in .jpeg or .svg formats. Only returned for Mini Apps launched from the attachment menu.

Validating data received via the Mini App

To validate data received via the Mini App, one should send the data from the *Telegram.WebApp.initData* field to the bot's backend. The data is a query string, which is composed of a series of field-value pairs.

You can verify the integrity of the data received by comparing the received *hash* parameter with the hexadecimal representation of the HMAC-SHA-256 signature of the **data-check-string** with the secret key, which is the HMAC-SHA-256 signature of the bot's token with the constant string `WebAppData` used as a key.

Data-check-string is a chain of all received fields, sorted alphabetically, in the format `key=<value>` with a line feed character ('
', 0x0A) used as separator – e.g., `'auth_date=<auth_date> query_id=<query_id> user=<user>'`.

The full check might look like:

```
data_check_string = ...
secret_key = HMAC_SHA256(<bot_token>, "WebAppData")
if (hex(HMAC_SHA256(data_check_string, secret_key)) == hash) {
  // data is from Telegram
}
```

To prevent the use of outdated data, you can additionally check the *auth_date* field, which contains a Unix timestamp of when it was received by the Mini App.

Once validated, the data may be used on your server. Complex data types are represented as JSON-serialized objects.

Validating data for Third-Party Use

If you need to share the data with a third party, they can validate the data without requiring access to your bot's token. Simply provide them with the data from the *Telegram.WebApp.initData* field and your *bot_id*.

The integrity of the data can be verified by validating the received *signature* parameter, which is the base64url-encoded representation of the Ed25519 signature of the **data-check-string**. The verification is performed using the public key provided by Telegram.

Data-check-string is constructed as follows:
1. Prepend the *bot_id*, followed by `:` and the constant string `WebAppData`.
2. Add a line feed character (`'\n'`, 0x0A).
3. Append all received fields (except *hash* and *signature*), sorted alphabetically, in the format `key=<value>`.
4. Separate each key-value pair with a line feed character (`'\n'`, 0x0A).

Example:
`'12345678:WebAppData auth_date=<auth_date> query_id=<query_id> user=<user>'`

The verification process might look like this:

```
data_check_string = ...
public_key = "<Telegram_public_key>"
if (Ed25519_verify(public_key, data_check_string, signature)) {
  // data is valid and originated from Telegram
}
```

> Telegram provides the following Ed25519 public keys for signature verification:
>
> **Test environment:** `40055058a4ee38156a06562e52eece92a771bcd8346a8c4615cb7376eddf72ec` (hex)
> **Production:** `e7bf03a2fa4602af4580703d88dda5bb59f32ed8b02a56c187fe7d34caed242d` (hex)

To prevent the use of outdated data, the third party should additionally validate the *auth_date* field. This field contains a Unix timestamp indicating when the data was received by the Mini App.

Once validated, the data may be used. Complex data types are represented as JSON-serialized objects.

Events Available for Mini Apps

The Mini App can receive events from the Telegram app, onto which a handler can be attached using the `Telegram.WebApp.onEvent(eventType, eventHandler)` method. Inside `eventHandler` the *this* object refers to *Telegram.WebApp*, the set of parameters sent to the handler depends on the event type. Below is a list of possible events:

eventType	Description
`activated`	Bot API 8.0+ Occurs when the Mini App becomes active (e.g., opened from minimized state or selected among tabs). *eventHandler* receives no parameters.
`deactivated`	Bot API 8.0+ Occurs when the Mini App becomes inactive (e.g., minimized or moved to an inactive tab). *eventHandler* receives no parameters.
`themeChanged`	Occurs whenever theme settings are changed in the user's Telegram app (including switching to night mode). *eventHandler* receives no parameters, new theme settings and color scheme can be received via *this.themeParams* and *this.colorScheme* respectively.

eventType	Description
`viewportChanged`	Occurs when the visible section of the Mini App is changed. *eventHandler* receives an object with the single field *isStateStable*. If *isStateStable* is true, the resizing of the Mini App is finished. If it is false, the resizing is ongoing (the user is expanding or collapsing the Mini App or an animated object is playing). The current value of the visible section's height is available in *this.viewportHeight*.
`safeAreaChanged`	Bot API 8.0+ Occurs when the device's safe area insets change (e.g., due to orientation change or screen adjustments). *eventHandler* receives no parameters. The current inset values can be accessed via *this.safeAreaInset*.
`contentSafeAreaChanged`	Bot API 8.0+ Occurs when the safe area for content changes (e.g., due to orientation change or screen adjustments). *eventHandler* receives no parameters. The current inset values can be accessed via *this.contentSafeAreaInset*.
`mainButtonClicked`	Occurs when the main button is pressed. *eventHandler* receives no parameters.
`secondaryButtonClicked`	Bot API 7.10+ Occurs when the secondary button is pressed. *eventHandler* receives no parameters.
`backButtonClicked`	Bot API 6.1+ Occurrs when the back button is pressed. *eventHandler* receives no parameters.
`settingsButtonClicked`	Bot API 6.1+ Occurrs when the Settings item in context menu is pressed. *eventHandler* receives no parameters.
`invoiceClosed`	Bot API 6.1+ Occurrs when the opened invoice is closed. *eventHandler* receives an object with the two fields: *url* – invoice link provided and *status* – one of the invoice statuses: - **paid** – invoice was paid successfully, - **cancelled** – user closed this invoice without paying, - **failed** – user tried to pay, but the payment was failed, - **pending** – the payment is still processing. The bot will receive a service message about a successful payment when the payment is successfully paid.
`popupClosed`	Bot API 6.2+ Occurrs when the opened popup is closed. *eventHandler* receives an object with the single field *button_id* – the value of the field *id* of the pressed button. If no buttons were pressed, the field *button_id* will be *null*.
`qrTextReceived`	Bot API 6.4+ Occurs when the QR code scanner catches a code with text data. *eventHandler* receives an object with the single field *data* containing text data from the QR code.
`scanQrPopupClosed`	Bot API 7.7+ Occurs when the QR code scanner popup is closed by the user. *eventHandler* receives no parameters.
`clipboardTextReceived`	Bot API 6.4+ Occurrs when the `readTextFromClipboard` method is called. *eventHandler* receives an object with the single field *data* containing text data from the clipboard. If the clipboard contains non-text data, the field *data* will be an empty string. If the Mini App has no access to the clipboard, the field *data* will be *null*.
`writeAccessRequested`	Bot API 6.9+ Occurs when the write permission was requested. *eventHandler* receives an object with the single field *status* containing one of the statuses: - **allowed** – user granted write permission to the bot, - **cancelled** – user declined this request.

eventType	Description
`contactRequested`	Bot API 6.9+ Occurrs when the user's phone number was requested. *eventHandler* receives an object with the single field *status* containing one of the statuses: - **sent** – user shared their phone number with the bot, - **cancelled** – user declined this request.
`biometricManagerUpdated`	Bot API 7.2+ Occurs whenever BiometricManager object is changed. *eventHandler* receives no parameters.
`biometricAuthRequested`	Bot API 7.2+ Occurs whenever biometric authentication was requested. *eventHandler* receives an object with the field *isAuthenticated* containing a boolean indicating whether the user was authenticated successfully. If *isAuthenticated* is true, the field *biometricToken* will contain the biometric token stored in secure storage on the device.
`biometricTokenUpdated`	Bot API 7.2+ Occurs whenever the biometric token was updated. *eventHandler* receives an object with the single field *isUpdated*, containing a boolean indicating whether the token was updated.
`fullscreenChanged`	Bot API 8.0+ Occurs whenever the Mini App enters or exits fullscreen mode. *eventHandler* receives no parameters. The current fullscreen state can be checked via *this.isFullscreen*.
`fullscreenFailed`	Bot API 8.0+ Occurs if a request to enter fullscreen mode fails. *eventHandler* receives an object with the single field *error*, describing the reason for the failure. Possible values for *error* are: **UNSUPPORTED** – Fullscreen mode is not supported on this device or platform. **ALREADY_FULLSCREEN** – The Mini App is already in fullscreen mode.
`homeScreenAdded`	Bot API 8.0+ Occurs when the Mini App is successfully added to the home screen. *eventHandler* receives no parameters.
`homeScreenChecked`	Bot API 8.0+ Occurs after checking the home screen status. *eventHandler* receives an object with the field *status*, which is a string indicating the current home screen status. Possible values for *status* are: - **unsupported** – the feature is not supported, and it is not possible to add the icon to the home screen, - **unknown** – the feature is supported, and the icon can be added, but it is not possible to determine if the icon has already been added, - **added** – the icon has already been added to the home screen, - **missed** – the icon has not been added to the home screen.
`accelerometerStarted`	Bot API 8.0+ Occurs when accelerometer tracking has started successfully. *eventHandler* receives no parameters.
`accelerometerStopped`	Bot API 8.0+ Occurs when accelerometer tracking has stopped. *eventHandler* receives no parameters.
`accelerometerChanged`	Bot API 8.0+ Occurs with the specified frequency after calling the `start` method, sending the current accelerometer data. *eventHandler* receives no parameters, the current acceleration values can be received via *this.x*, *this.y* and *this.z* respectively.
`accelerometerFailed`	Bot API 8.0+ Occurs if a request to start accelerometer tracking fails. *eventHandler* receives an object with the single field *error*, describing the reason for the failure. Possible values for *error* are: **UNSUPPORTED** – Accelerometer tracking is not supported on this device or platform.

eventType	Description
`deviceOrientationStarted`	Bot API 8.0+ Occurs when device orientation tracking has started successfully. *eventHandler* receives no parameters.
`deviceOrientationStopped`	Bot API 8.0+ Occurs when device orientation tracking has stopped. *eventHandler* receives no parameters.
`deviceOrientationChanged`	Bot API 8.0+ Occurs with the specified frequency after calling the `start` method, sending the current orientation data. *eventHandler* receives no parameters, the current device orientation values can be received via *this.alpha*, *this.beta* and *this.gamma* respectively.
`deviceOrientationFailed`	Bot API 8.0+ Occurs if a request to start device orientation tracking fails. *eventHandler* receives an object with the single field *error*, describing the reason for the failure. Possible values for *error* are: **UNSUPPORTED** – Device orientation tracking is not supported on this device or platform.
`gyroscopeStarted`	Bot API 8.0+ Occurs when gyroscope tracking has started successfully. *eventHandler* receives no parameters.
`gyroscopeStopped`	Bot API 8.0+ Occurs when gyroscope tracking has stopped. *eventHandler* receives no parameters.
`gyroscopeChanged`	Bot API 8.0+ Occurs with the specified frequency after calling the `start` method, sending the current gyroscope data. *eventHandler* receives no parameters, the current rotation rates can be received via *this.x*, *this.y* and *this.z* respectively.
`gyroscopeFailed`	Bot API 8.0+ Occurs if a request to start gyroscope tracking fails. *eventHandler* receives an object with the single field *error*, describing the reason for the failure. Possible values for *error* are: **UNSUPPORTED** – Gyroscope tracking is not supported on this device or platform.
`locationManagerUpdated`	Bot API 8.0+ Occurs whenever LocationManager object is changed. *eventHandler* receives no parameters.
`locationRequested`	Bot API 8.0+ Occurs when location data is requested. *eventHandler* receives an object with the single field *locationData* of type LocationData, containing the current location information.
`shareMessageSent`	Bot API 8.0+ Occurs when the message is successfully shared by the user. *eventHandler* receives no parameters.
`shareMessageFailed`	Bot API 8.0+ Occurs if sharing the message fails. *eventHandler* receives an object with the single field *error*, describing the reason for the failure. Possible values for *error* are: **UNSUPPORTED** – The feature is not supported by the client. **MESSAGE_EXPIRED** – The message could not be retrieved because it has expired. **MESSAGE_SEND_FAILED** – An error occurred while attempting to send the message. **USER_DECLINED** – The user closed the dialog without sharing the message. **UNKNOWN_ERROR** – An unknown error occurred.
`emojiStatusSet`	Bot API 8.0+ Occurs when the emoji status is successfully set. *eventHandler* receives no parameters.

eventType	Description
`emojiStatusFailed`	Bot API 8.0+ Occurs if setting the emoji status fails. *eventHandler* receives an object with the single field *error*, describing the reason for the failure. Possible values for *error* are: **UNSUPPORTED** – The feature is not supported by the client. **SUGGESTED_EMOJI_INVALID** – One or more emoji identifiers are invalid. **DURATION_INVALID** – The specified duration is invalid. **USER_DECLINED** – The user closed the dialog without setting a status. **SERVER_ERROR** – A server error occurred when attempting to set the status. **UNKNOWN_ERROR** – An unknown error occurred.
`emojiStatusAccessRequested`	Bot API 8.0+ Occurs when the write permission was requested. *eventHandler* receives an object with the single field *status* containing one of the statuses: - **allowed** – user granted emoji status permission to the bot, - **cancelled** – user declined this request.
`fileDownloadRequested`	Bot API 8.0+ Occurs when the user responds to the file download request. *eventHandler* receives an object with the single field *status* containing one of the statuses: - **downloading** – the file download has started, - **cancelled** – user declined this request.

Adding Bots to the Attachment Menu

Attachment menu integration is currently only available for major advertisers on the Telegram Ad Platform. However, **all bots** can use it in the test server environment. Talk to Botfather on the test server to set up the integration.

A special link is used to add bots to the attachment menu:

`https://t.me/botusername?startattach`
or
`https://t.me/botusername?startattach=command`

> For example, open this attachment menu link for *@DurgerKingBot*, then use the  menu in any **private chat**.

Opening the link prompts the user to add the bot to their attachment menu. If the bot has already been added, the attachment menu will open in the current chat and redirect to the bot there (if the link is opened from a 1-on-1 chat). If a non-empty *startattach* parameter was included in the link, it will be passed to the Mini App in the *start_param* field and in the GET parameter *tgWebAppStartParam*.

The following link formats are also supported:

`https://t.me/username?attach=botusername`
`https://t.me/username?attach=botusername&startattach=command`
`https://t.me/+1234567890?attach=botusername`
`https://t.me/+1234567890?attach=botusername&startattach=command`

These links open the Mini App in the attachment menu in the chat with a specific user. If the bot wasn't already added to the attachment menu, the user will be prompted to do so. If a non-empty *startattach* parameter was included in the link, it will be passed to the Mini App in the *start_param* field and in the GET parameter *tgWebAppStartParam*.

Bot API 6.1+ supports a new link format:

`https://t.me/botusername?startattach&choose=users+bots`
`https://t.me/botusername?startattach=command&choose=groups+channels`

Opening such a link prompts the user to choose a specific chat and opens the attachment menu in that chat. If the bot wasn't already added to the attachment menu, the user will be prompted to do so. You can specify which types of chats the user will be able to choose from. It can be one or more of the following types: *users*, *bots*, *groups*, *channels* separated by a `+` sign. If a non-empty *startattach* parameter was included in the link, it will be passed to the Mini App in the *start_param* field and in the GET parameter *tgWebAppStartParam*.

Additional Data in User-Agent

When the Mini App is running on Android, additional information is appended to the User-Agent string to provide more context about the app environment. This information includes the app version, device model, Android version, SDK version, and device performance class, formatted as follows:

```
Telegram-Android/{app_version} ({manufacturer} {model}; Android {android_version}; SDK {sdk_version}; {performance_class})
```

where:

- **{app_version}** is the version of the Telegram app (e.g., `11.3.3`),
- **{manufacturer} {model}** represents the device's manufacturer and model (e.g., `Google sdk_gphone64_arm64`),
- **{android_version}** is the Android OS version running on the device (e.g., `14`),
- **{sdk_version}** indicates the Android SDK version (e.g., `34`),
- **{performance_class}** specifies the device performance class as `LOW`, `AVERAGE`, or `HIGH`, indicating the device's performance capacity.

> **Example**
> `Mozilla/5.0 (Linux; Android 14; K) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/113.0.5672.136 Mobile Safari/537.36 Telegram-Android/11.3.3 (Google sdk_gphone64_arm64; Android 14; SDK 34; LOW)`

We recommend using this information to optimize your Mini App based on the device's capabilities. For instance, you can adjust animations and visual effects in games on low-performance devices to ensure a smooth experience for all users, regardless of device specifications.

Testing Mini Apps

Using bots in the test environment

To log in to the test environment, use either of the following:

- **iOS:** tap 10 times on the Settings icon > Accounts > Login to another account > Test.
- **Telegram Desktop:** open ☰ Settings > Shift + Alt + Right click 'Add Account' and select 'Test Server'.
- **macOS:** click the Settings icon 10 times to open the Debug Menu, ⌘ + click 'Add Account' and log in via phone number.

The test environment is completely separate from the main environment, so you will need to create a **new user account** and a **new bot** with @BotFather.

After receiving your bot token, you can send requests to the Bot API in this format:

```
https://api.telegram.org/bot<token>/test/METHOD_NAME
```

> **Note:** When working with the test environment, you may use HTTP links without TLS to test your Mini App.

Debug Mode for Mini Apps

Use these tools to find app-specific issues in your Mini App:

iOS

- In Telegram tap 10 times on the Settings icon and toggle on *Allow Web View Inspection*.
- Connect your phone to your computer using a USB cable.
- Open Safari on your Mac, then go to *Develop > [Your Device Name]* in the menu bar.
- Launch your Mini App on the iOS device – it will appear in the *Develop* menu under your device.

Android

- Enable USB-Debugging on your device.
- In Telegram Settings, scroll all the way down, press and hold on the **version number** two times.
- Choose *Enable WebView Debug* in the Debug Settings.
- Connect your phone to your computer and open `chrome://inspect/#devices` in Chrome – you will see your Mini App there when you launch it on your phone.

Telegram Desktop on Windows and Linux

- Download and launch the Beta Version of Telegram Desktop on **Windows** or **Linux** (not supported on Telegram Desktop for macOS yet).
- Go to *Settings > Advanced > Experimental settings > Enable webview inspection*.
- Right click in the WebView and choose *Inspect*.

Telegram macOS

- Download and launch the Beta Version of Telegram macOS.
- Quickly click 5 times on the Settings icon to open the debug menu and enable "Debug Mini Apps".

- Right click in the Mini App and choose *Inspect Element*.